Laboratorijska vježba 4

HNSW

Napredni algoritmi i strukture podataka

Opis vježbe

Za 4. laboratorijsku vježbu zadatak vam je proučiti i implementirati strukturu podataka za približno pretraživanje najbližih susjeda **HNSW**. Ta struktura pokušava zaobići probleme koji se pojave kod pretrage velikog broja točaka u visokim dimenzijama koristeći lokalnu strukturu podataka (koja srećom nije uvijek uniformna). Cilj je da sami istražite i sagradite intuiciju o ponašanju HNSW-a, tj. kada i zašto promaši najbliže točke. Opis algoritma nalazi se u knjizi u odjeljku 4.8, a popratni kod u datoteci hnsw.py na popratnom GitHub repozitoriju.

Sama struktura sastoji se od hijerarhije grafova (što je opet jedan graf), gdje je razina 0 najgušće popunjen graf susjedstva, a više razine služe za bržu pretragu između različitih susjedstava. Način na koji spremate grafove po razinama vam je otvoren, tj. možete birati sami strukturu podataka koju želite koristiti za spremanje grafova i svega ostalog što nije eksplicitno navedeno.

Za samu vježbu osim opisa traženog ponašanja, dobivate i skupove podataka (odjeljak *Podatci*, str. 4) koji će vam služiti za provjeru i izgradnju intuicije kroz eksperimentaciju.

Parametri

Vaša implementacija mora omogućiti prosljeđivanje sljedećih parametara:

naziv	značenje	u knjizi
`max_neighbors`	Najveći broj susjeda točke na razinama $l > 0$; za razinu $l = 0$ vrijednost iznosi $2 \times$ max_neighbors.	M
`num_build_candidates`	Broj kandidata koji se razmatraju pri dodavanju nove točke na svakoj razini.	`ef_construction`
`num_candidates`	Broj kandidata koji se razmatraju pri pretrazi na razini 0, od kojih se vraća `num_results` najbližih.	`ef`
`num_results`	Broj traženih najbližih točaka.	k
`selection_rule`	Pravilo odabira susjeda (`diverse` ili `nearest`; vidi *Pravilo odabira susjeda*, str. 2).	4.8.1
`num_layers`	Broj razina: razina svake točke je najviše `num_layers` -1. Uz 1 sve su točke na razini 0 (graf bez razina).	uz 1: `flat`

Razlike od knjige

Vaš program od knjige se razlikuje u pet stvari:

1. pretraga susjede točke obilazi uzlazno po id-u
2. razina točke računa se iz broja r zadanog uz točku, (**ne koristite pseudo-slučajni generator**, formula je u *Implementacijskim pravilima*).
3. pravilo odabira susjeda i broj razina zadaju se parametrima `selection_rule` i `num_layers` (vidi *Parametri* i *Pravilo odabira susjeda*).
4. pretraga broji koliko je različitih točaka posjećeno.
5. uz svaki upit program zna reći koje od najbližih točaka nije pronašao i zašto. Za to razlikujemo dvije vrste pogreške:
   a) *„Promašene"* točke – pretraga ih nije vratila, ali do njih vodi put po vezama razine 0 (od točke prema njezinim susjedima) od čvora na kojem je pretraga ušla u razinu 0.

b) *„Nedostupne"* točke – nema takvog puta na razini 0 do njih.

Implementacijska pravila

Uz razlike, fiksirat ćemo nekoliko implementacijskih pravila radi jednoznačnosti rezultata:

- Udaljenost točaka p i q je
$$d(p,q) = \sum_{i=1}^{\texttt{dim}} (p_i - q_i)^2,$$
bez korijena, gdje je `dim` broj koordinata.
- Koordinate i udaljenosti su cijeli brojevi.
- Točke dobivaju id-ove $0, 1, 2, \ldots$ redom dodavanja.
- Pri jednakoj udaljenosti bliža je točka s manjim id-om.
- Razina točke je
$$\ell = \max\{\ell \geq 0 : r \cdot M^\ell \leq 10^6\}, \qquad M = \texttt{max_neighbors},$$
gdje je r slučajni broj zadan uz točku. On zamjenjuje slučajni broj kojim HNSW bira razinu. Računajte razinu cijelim brojevima, kao funkcija `level` u odjeljku *Dodatak* (str. 6), napisana u svakom od dozvoljenih jezika. Točka zatim dobiva razinu $\min(\ell, \texttt{num_layers} - 1)$.

Pravilo odabira susjeda

Parametar `selection_rule` određuje kako točka bira susjede od kandidata. Bira se u dva slučaja:

- **dodavanje točke:** na svakoj svojoj razini točka bira najviše m = `max_neighbors` susjeda (i na razini 0) od kandidata koje je našla pretraga;
- **ažuriranje susjeda postojeće točke:** kad točka na nekoj razini ima više susjeda nego što smije, ponovno odabire susjede. Uz to, m = `max_neighbors` iznad razine 0 i $m = 2 \times$ `max_neighbors` na razini 0.

Neka je v trenutna točka za koju biramo susjede, a potencijalni susjedi poredani silazno od najbližeg točki v. Parametar ima dvije vrijednosti:

- `diverse` (*pravilo raznolikosti*, odjeljak 4.8.1): kandidati se gledaju redom. Kandidat c se odabire ako za svakog već odabranog susjeda s vrijedi
$$d(c,v) \leq d(c,s),$$
tj. ako mu nijedan odabrani susjed nije strogo bliži nego v; inače se preskače. Staje se kad je odabrano m susjeda ili kad kandidata nestane, pa ih može biti i manje od m. Ideja: do kandidata koji je bliži već odabranom susjedu nego točki v dolazi se i preko tog susjeda, pa veza do njega ne donosi novi smjer.
- `nearest` (*samo najbliži*, pohlepni odabir): odabire se m najbližih kandidata, bez ikakve provjere.

Razliku dvaju pravila na istim točkama pokazuje kraj odjeljka *Primjer* (str. 2).

Podatci za provjeru implementacije i eksperimentaciju opisani su u odjeljku *Podatci*.

Primjer

Dvanaest točaka iz knjige (odjeljak 4.8), dodanih redom od `a` do `l`, uz `max_neighbors` = 2, `num_build_candidates` = 16 i `num_layers` = 3 (razine 0–2, kao u knjizi).

točka	id	koordinate	r	razina
a	0	(2, 15)	400000	1
b	1	(17, 19)	260000	1
c	2	(4, 20)	333333	1
d	3	(6, 6)	150000	2
e	4	(18, 1)	700000	0
f	5	(9, 18)	820000	0
g	6	(1, 10)	410000	1
h	7	(3, 9)	280000	1
i	8	(15, 8)	555555	0
j	9	(19, 3)	490000	1
k	10	(8, 5)	910000	0
l	11	(10, 17)	640000	0

točka	susjedi na razini 0	na razini 1
a	b c d g	c g
b	a c f l	c j
c	a b f	a b
d	h i k	h j
e	d i j	
f	b c l	
g	a d h	a h
h	d g	d g
i	d e j k	
j	e i	b d
k	d i	
l	b f	

Kao u knjizi (odjeljak 4.8.4), ulazna točka je `d` (id 3), jedina na razini 2. Uz `num_layers` = 1 graf na razini 0 ima iste veze, a pretraga kreće od `a` (id 0). Slova su dodana radi čitljivosti, program ispisuje id-ove.

Pretraga za točku upita $q = (12, 0)$, kojoj je najbliža točka `e` (udaljenost 37):

`num_layers`	`num_candidates`	rezultat	posjećeno točaka
1	1	`k` (41); `e` je promašena, iako do nje vodi put	8
1	3	`e k j`	10
3	1	`e`: `d` na razini 2, `j` na razini 1, `e` na razini 0	6
3	3	`e k j`	7

Posjećeno točaka je broj različitih točaka do kojih je pretraga izračunala udaljenost (skup `visited` u kodu knjige), uključujući ulaznu točku. Iste pretrage opisane su u knjizi (odjeljci 4.8.2 i 4.8.4):

- `num_layers` = 1, **jedan kandidat:** pretraga kreće od `a`, prelazi u `d`, pa u `k` i ondje staje, jer nijedan susjed točke `k` nije bliži točki q.
- Najbliža točka `e` pritom je *promašena*: do nje vodi put `a` $\to$ `d` $\to$ `i` $\to$ `e`. U točki `d` pretraga je posjetila i točku `i` (udaljenost 73), ali `k` je bliža (41), pa je s jednim kandidatom otišla u `k`. S tri kandidata pretraga zadrži i točku `i`, prođe kroz nju te nađe `e`.
- `num_layers` = 3, **jedan kandidat:** pretraga kreće od `d` na razini 2, na razini 1 prelazi u `j`, a na razini 0 iz `j` prelazi u `e`.

Pravila odabira. Sve gore vrijedi uz `selection_rule = diverse`. Uz `nearest`, na istim točkama i s istim parametrima:

	`diverse`	`nearest`
susjedi točke `d` na razini 0 nakon dodavanja točke `i`	`h, i`	`a, g, h, i`
pretraga s jednim kandidatom uz `num_layers` = 3 za točku upita (12, 0) vraća	`e`	`k`

Stupac `diverse` je primjer iz knjige (odjeljak 4.8.3). Uz samo najbliže točka `d` zadrži četiri najbliža susjeda, pa ista pretraga staje u `k`, a ne u `e`.

Literatura

- Dizajn 4.4: invarijante i pretpostavka HNSW-a
- 4.8 Ask the locals: odabir susjeda (4.8.1), pretraga (4.8.2), dodavanje (4.8.3), razine (4.8.4), trošak (4.8.5) i 4.8.6 What it cannot promise
- 4.13.6 Fix the code, the box, or the design?
- Kod knjige (repozitorij `mbrcic/algodesign`): `python/algodesign/geometry/hnsw.py`

Podatci

Uz pripremu dobivate i nekoliko datoteka za provjeru vaše implementacije i za provođenje eksperimenata. Datoteke u svakom retku sadrže koordinate i zatim pseudo-slučajan broj za unos točke. Prema tome, za datoteke sa dvije koordinate, svaki redak sastoji se od $x_1 \quad x_2 \quad r$.

Dobivate i datoteke koje sadrže primjere upita, one sadrže samo koordinate točaka koje pretražujemo. Uz svaku datoteku upita imate i datoteke očekivanih rezultata.

Na vježbi će program učitavati datoteke s točkama za unos, a testovi će zatim dodavati i nove točke.

datoteka	sadržaj
`book-12.txt`	12 točaka iz primjera, za provjeru na papiru
`clumps-8.txt`	8 točaka u tri skupine, za provjeru na papiru
`uniform-`*N*`.txt`	*N* = 400, 800, 1600, 3200 ravnomjerno raspoređenih točaka
`clustered-`*N*`.txt`	*N* = 400, 800, 1600, 3200 točaka u skupinama
`uniform-queries.txt`	200 upita za `uniform`, isti za sve *N*
`clustered-queries.txt`	200 upita za `clustered`, isti za sve *N*
`clustered32-`*N*`.txt`	veliki skup (zasebna arhiva): točke s 32 koordinate u skupinama, *N* = 5000, 10000, 20000
`clustered32-queries.txt`	1000 upita
`clustered32-`*N*`-truth.txt`	10 točaka najbližih točki svakog upita (id-ovi, od najbliže)

Id-ovi. Točka dobiva id redom dodavanja, od 0: u učitanoj datoteci id je redni broj retka (prvi redak je id 0). Ako se učita još jedna datoteka ili doda još neka točka, brojanje se nastavlja.

Primjer najmanje datoteke `clumps-8.txt` (brojevi lijevo su id-ovi i nisu dio datoteke):

clumps-8.txt

```
0 13 6 700000
1 8 11 700000
2 8 14 700000
3 24 38 700000
4 21 40 700000
5 7 8 700000
6 9 12 700000
7 44 13 700000
```

Svaki redak: koordinate *x* i *y*, a zatim slučajni broj *r* za razinu. Ovdje sve točke imaju isti *r*, pa su sve na razini 0 za svaki `max_neighbors` $\geq$ 2. U datoteci s upitima retci su samo koordinate, npr. `26 39`.

Istražite sami

Ako nije drukčije rečeno, postavke su `max_neighbors` = 4, `num_build_candidates` = 16, `num_results` = 10, `num_candidates` = 10, `selection_rule` = `diverse` i `num_layers` = 20 (tj. bez ograničenja: razinu svake točke određuje samo *r*). Za svaku postavku bilježite, prosječno po upitu, *udio pronađenih najbližih točaka* (koliko od `num_results` najbližih točaka pretraga vrati) i *broj posjećenih točaka*.

1. **Na papiru** (`book-12.txt`). Dodajte točku `i` kao u knjizi (odjeljak 4.8.3), s ponovnim odabirom susjeda točke `d`, pa reproducirajte tablice iz primjera. *Cilj:* pravilo odabira, ponovni odabir i razine.
2. **Mjerenje složenosti i pogrešaka** (`uniform-`*N*, `clustered-`*N* i njihovi upiti). Usporedite tri postavke: zadane (*default*), zadane uz `selection_rule` = `nearest` i zadane uz `num_layers` = 1. Za svaku postavku i svaki *N* izmjerite udio pronađenih najbližih točaka i broj posjećenih točaka, pa oba broja nacrtajte u ovisnosti o *N*. *Cilj:* što donosi koji dio dizajna.
3. **Vrste pogrešaka** (`clumps-8.txt`, na papiru). Najbliže točke koje pretraga nije vratila podijelite na *promašene* i *nedostupne* (vidi *Razlike od knjige*, str. 1). *Cilj:* koja postavka daje koju vrstu pogreške.
4. **Smanjivanje pogrešaka.** Mijenjajte broj kandidata pri pretrazi (`num_candidates`), broj kandidata pri gradnji (`num_build_candidates`), najveći broj susjeda (`max_neighbors`), pravilo odabira

(`selection_rule`) i broj razina (`num_layers`), najprije jedan po jedan, a zatim u kombinacijama (npr. veći `num_build_candidates` uz manji `num_candidates`). *Cilj:* koja promjena ili kombinacija smanjuje koju vrstu pogreške i uz koliko više posjećenih točaka.

5. **Broj razina** (`uniform-`*N*, `clustered-`*N*). Ponovite mjerenje složenosti i pogrešaka za `num_layers` = 1, 2, 3, … dok se rezultat ne prestane mijenjati. *Cilj:* kako broj razina utječe na pogreške i na broj posjećenih točaka, i ovisi li to o obliku podataka.
6. **Pokušajte potrgati strukturu** (`clustered-400.txt`, `uniform-400.txt`). Učitajte točke iz datoteke i provedite odgovarajuće upite. Zatim dodajte nove točke: gustu skupinu daleko od svih podataka, niz točaka na pravcu preko cijelog prostora ili mnogo točaka jednakih koordinata. Ponovite stare upite i postavite nove, pogotovo blizu novo dodanih točaka. *Cilj:* koja se pogreška pojavi, uz koje postavke i zašto. Dio otkrivenih testova na vježbi radi upravo ovo na ovim datotekama, a skriveni testovi isto na drugim podatcima.
7. **Veliki skupovi** (`clustered32-`*N*). Ponovite pokuse 2, 4 i 5 te usporedite broj posjećenih točaka s pretraživanjem svih točaka. Vrijeme možete izmjeriti, ali ono ovisi o vašem računalu i jeziku. *Cilj:* što se mijenja s više dimenzija.

Za razmišljanje

- Kako nastaje točka do koje ne vodi put, iako je pri dodavanju svaki njezin odabrani susjed doda među svoje susjede?
- Što očekujete za dvodimenzionalni `clustered` s 2400 i s 6400 točaka? Što od izmjerenoga možete predvidjeti, a što ne?
- Kada redoslijed kojim pretraga obilazi susjede mijenja rezultat, a kada ne?
- Kako spremate graf? Uz strukture podataka koje ste odabrali, koliko je složen ponovni odabir, a koliko provjera do kojih točaka vodi put?
- Koji dio dizajna sprječava koju pogrešku? Što od toga obećava dizajn 4.4, a što se samo mjeri?
- Poglavlje opisuje i stabla raspona (*range trees*, odjeljak 4.2) i k-d stabla (odjeljak 4.3). Kako mislite da bi se ona ponašala na skupovima `clustered32-`*N* (32 dimenzije) u usporedbi s HNSW-om: koliko bi točaka posjetila, koliko bi memorije trebala i koliko bi točni bili odgovori?

Kako teče vježba

Na vježbi dobivate točan zadatak i svoje pripremljeno rješenje prilagođavate njemu. Program čita sa standardnog ulaza, piše na standardni izlaz i izvodi se na skrivenim testovima, koji uspoređuju ispis nakon svake naredbe. Imate **2 sata** da program prođe sve testove i da započnete kviz. Kviz o svojem rješenju, pokusima i gradivu otključava se **tek kada program prođe sve testove**, traje **10 minuta** i ima **samo jedan pokušaj**.

Dozvoljeni jezici

Jezik	Inačica i opcije prevođenja
Python	3.12.15
C	GCC 15.2.0, `-O2`
C++	GCC 15.2.0, `-O2`, `-std=c++23`
Rust	1.98.1, `--edition=2021`, `opt-level=3`
Java	OpenJDK 21.0.2

Dodatak

Funkcija `level` daje razinu točke sa slučajnim brojem r, $\ell = \max\{\ell \geq 0 : r \cdot M^\ell \leq 10^6\}$, u svakom od dozvoljenih jezika; parametar `m` je M = `max_neighbors`.

Python

```
def level(r, m):
    lvl, p = 0, m
    while r * p <= 1_000_000:
        lvl += 1
        p *= m
    return lvl
```

C i C++

```
int level(long long r, long long m) {
    int lvl = 0;
    for (long long p = m; r * p <= 1000000;
         p *= m)
        lvl++;
    return lvl;
}
```

Java

```
static int level(long r, long m) {
    int lvl = 0;
    for (long p = m; r * p <= 1_000_000;
         p *= m)
        lvl++;
    return lvl;
}
```

Rust

```
fn level(r: i64, m: i64) -> u32 {
    let mut lvl = 0;
    let mut p = m;
    while r * p <= 1_000_000 {
        lvl += 1;
        p *= m;
    }
    lvl
}
```